

计算机视觉 (CV)

Computer Vision

Author: Simson

Date: September 2026

目录

1 导论	1
2 VCL 基础知识和技能	1
2.1 颜色, 颜色感知和可视化	1
3 显示 (Display)	1
3.1 显示器的种类和发展历史	1
3.2 色域	2
3.3 像素标准	2
3.4 立体显示	2
4 图像 (Image Representation)	2
4.1 图像的表达	3
4.2 图形处理器 (GPU)	3
4.3 图像处理 (Image Processing)	4
4.3.1 Dithering	4
4.3.2 Filtering	5
4.3.3 Image inpainting	6

1 导论

可视计算的定义：针对视觉内容生成与处理的所有计算机课学分制的总称（计算机图形学，计算机视觉，可视化，AR,VR）人类智能在感知，认知，决策，行动，感知的循环中逐渐积累。

重要课题：

- 物理世界仿真与模拟
- 计算机图形学
- 计算机视觉

2 VCL 基础知识和技能

2.1 颜色，颜色感知和可视化

人感知颜色靠的是对不同波长的光敏感的三种视锥细胞，这提供了颜色离散表达的基础。三种感光开展出 RGB 三色系统。颜色的匹配函数如下，是一个积分的办法。

$$\begin{aligned}R &= \int_0^{\infty} I(\lambda) \bar{r}(\lambda) d\lambda \\G &= \int_0^{\infty} I(\lambda) \bar{g}(\lambda) d\lambda \\B &= \int_0^{\infty} I(\lambda) \bar{b}(\lambda) d\lambda\end{aligned}$$

三种通道和原色的光由离散合成有两种机制：基于发光叠加相加混光（显示器发光原理）和基于白光吸收的相减混光（涂料和打印原理）。

计算机视觉基于人对光的感知。在人的感知之下，红色有前进的效应，蓝色会有看起来后退的效应。此即所谓色彩立体视觉。同时在人的认知下颜色也有冷色和暖色之分（这真的是课堂 PPT 里面的）。颜色信息在人眼中具有恒常性，不论在何种光照之下人对于颜色的认知不会发生大的改变。

当然这种逻辑感知也会带来相应的视觉错觉，譬如色诱导，使用补色可以让人自动脑补对应的颜色；再比如说边缘效应和移动错觉（动态立体视觉）。

人眼对于不同的光敏感性不同，对不同波段的光，施加灰度和模糊处理可以得到不同的认知效果（灰度指的是介于黑白之间的明暗程度）。

3 显示 (Display)

3.1 显示器的种类和发展历史

1. 矢量显示（阴极射线管）
2. 二维显示（电子束）
 - 像素：每个可以点亮的屏幕点

- 分辨率：品目像素格式
 - 纵横比：像素列数除以行数
 - 帧：显示的一个画面
 - 隔行扫描：保证亮度均衡
3. 彩色二维显示（彩色荧光幕，不同荧光剂分区涂布）
 4. 二维显示（LCD）：统一发光，用液晶部分控制光的偏振，调节亮度
 5. 二维显示（OLED）：有机发光半导体显示器。独立的 LED 单元。

3.2 色域

美国国家电视标准委员会的色域标准称为 NTSC。

- sRGB：微软，HP（72）
- ARGB: Adobe 颜色（设计软件）
- Rec.2020:4K(电视的标准)
- DCI-P3 : Apple 标准

3.3 像素标准

显卡的存储和刷新越厉害，达到 4K8K 就越容易。

3.4 立体显示

立体显示依据的原理是双目视觉。所有的立体显示技术基于人类的生理特征。最开始的显示技术是双目偏振片的 3D 显示。沉浸式的 3D 显示技术（AR,VR,XR）跟踪头显，裸眼 3D+ 眼睛跟踪。现有的技术向 MR-mix 方向进行发展。

裸眼 3D 技术：LCD 屏幕的方法：微柱透镜显示技术。利用视差屏障，令 OLED 屏幕上不同的像素点用不同角度进入人眼。

全息显示技术（Holographic display）：用干涉记录波前的信息（数字全息摄影），用衍射记录物光波场（受限于较小的范围）。

多模态显示技术：触觉显示（触觉输出），嗅觉显示（控制挥发的成分。。。?）

4 图像（Image Representation）

在二维空间中，每个点可以由四个参数表示。 $E(x, y, \lambda, t)$ 其中 λ 字母的含义为光的波长。从数学上来说图像本身是一个连续的代表（真实世界），在屏幕上离散化显示。VCL 的任务在于用离散方式进行还原。

4.1 图像的表达

图像显示的两个方法是矢量表示法（Vector Representation，对应 SVG 格式）和光栅化表示法（Raster Representation，像素级存储）。

矢量表示法的精确度高，用较小的空间，不会受到分辨率影响（可以用新的矢量计算适应不同屏幕不同大小）多用于图标等规则图形。Raster 存储空间较大，受限于分辨率，但是可以表达更加不规则的图形。

4.2 图形处理器（GPU）

GPU 有成百上千个简单的计算单元，可以在同一时间并行运行大量程序，比如计算每个像素的颜色，而 CPU 一般只有个位数的可并行的计算单元。故而 GPU 显卡适合处理进行大量计算的图形编程任务。

Pixel 存储中有重要概念帧缓冲区（Frambuffers for image processing and display），指的是储存二维像素矩阵的内存大小。硬件以 60Hz 速度扫描帧缓冲区中的内容，这也决定了显示器的种类。

同样存储彩色图片的时候，Fullcolor（RGB），决定了显示器的颜色存储位数。一般来说真彩色图像的显示屏为 24-bit 位存储。如果颜色粗出空间较小，色彩的丰富度就会下降。

gif 颜色存储的性质：Colormap 映射。对于每一个像素点，存储一个 8-bit 常数，映射到 RGB 颜色表格 Colormap，根据映射进行显示。在预先选好的表格中进行查找和压缩。

几种图片的文件格式

常见图像文件格式的全称：

- **BMP**: Full color image（全彩色图像）
- **JPEG**: Joint Photographic Experts Group Format
- **TIFF**: Tagged-Image File Format
- **GIF**: CompuServe Graphics Interchange Format
- **PPM**: Portable PixMap Format（ASCII 或二进制）
- **EPS**: Encapsulated PostScript Format（ASCII）

	BITS PER PIXEL	FILE SIZE	COMMENTS
JPEG	24	small	lossy compression
TIFF	8, 24	medium	good general purpose
GIF	1, 4, 8	medium	popular, but 8-bit
PPM	24	big	easy to read/write
EPS	1, 2, 4, 8, 24	huge	good for printing

同时，相较于 24-bit 的图像，有的高级图像会膨胀到每个像素 96-bit，有一些其他的通道来表示图像。

1. Alpha 通道（8-bit）：透明度（Transparency），存储透明度常数，存在前中后背景。

根据参数 α 计算像素点的表示： $b = (1 - \alpha)a_1 + \alpha a_2$

2. Z-buffer (16-bit): 保存像素点所对应的 3D 空间点的深度。判断在叠加一层的时候应该占据多少通道和 Alpha 信息。
3. 双通道表示: 既要保存图像的现有数据, 又要保存图像的旧信息, 方便修改。

4.3 图像处理 (Image Processing)

图像处理按操作的作用范围可以分为两类基本操作。**点处理 (Point Processing)** 逐像素地修改像素值, 输出仅是该像素自身像素值的函数; **滤波 (Filtering)** 的输出则是该像素周围 (通常是局部邻域 (local neighborhood) 内像素的函数, 因而需要用到邻域中其他像素的信息。我们也可以理解为一个逐点操作, 一个是区域化的操作 (图像抖动, 图像过滤, 区域补全)。

除这两类基本操作之外, 图像处理还包括以下其他课题:

- 图像变换 (Image transformation): 缩放 (resize)、形变 (warp)
- 图像压缩 (Image compression): 无损 (lossless) 与有损 (lossy)
- 图像编解码 (Image encoding and decoding): 用于高效传输 (efficient transmission)
- 图像分割 (Image segmentation)
- 图像抠图 (Image matting)
- 图像理解 (Image understanding): 进一步走向计算机视觉 (computer vision)
- 视频处理 (Video processing): 推广到图像序列 (a sequence of images)
-

4.3.1 Dithering

灰度图像频率的定义 灰度图像频率可以理解为灰度数值变化的剧烈程度, 数学表示通过对像素矩阵进行 DFT (傅里叶变换)。

设 $f(x, y)$ 为 $M \times N$ 的灰度图像, 其二维 DFT 及逆变换为:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

其中 $F(u, v)$ 为频域系数 (j 为虚数单位), (u, v) 为频率坐标: u, v 靠近原点对应低频, 即灰度缓变区域; 远离原点对应高频, 即边缘和细节。变换本身不损失信息, 逆变换可无损还原 $f(x, y)$ 。

Dithering 和噪声种类 抖动和量化研究的是有限空间下图片的表示, 比如要把灰度图像表示成二值的图像。此时如果直接使用 0.5 为分界划分每个 pixel。这时要给整体图片接一个整体的随机噪音 (random dithering)。原来是结构化的容易被人察觉的低频误差转换成频谱较为均匀的高频误差。(所谓频率指的是低频区间的扰动)

施加噪音的方法有两种: Random Noise 和 Blue Noise, Blue Noise 将误差的能量集中到高频的空间, 低频范围内噪声较小, 高频噪声能量较高, 能够得到更好的表示效果。

噪声的施加如果作为输入, 还会用于图像的模糊处理 (Gaussian blur), 均匀处理掉细节的误差之后 Blue Noise 会有更好的效果。

Dithering in Space 用空间换表示效果: 可以用多个像素点空间表示不同灰度, 这时可以将抖动的幅度变成 1-10 的连续序列。所有的输入像素对应一个输出的 pattern。这里 0-9 的 9 像素显示模式是经过设计的, 为了人尽量难以注意到细小的细节错误。这种方法叫作 Ordered Dithering。

数学上的方法就是使用存储抖动矩阵记录抖动的序列, 依次按照输入的归一化数值点亮像素点。

如果保持连续性不变化, 不改变显示分辨率, 进行有序抖动, 可以用一个 Dither Matrix 将图片扫描一遍, 像素点大于抖动阵的数值则保留, 小于则纯白, 可以实现同样分布的抖动。

Error Diffusion 每一次抖动都会制造误差, 可以通过这样的算法让每一次的误差逐步扩散到周边像素。Floyd 算法可以决定抖动误差如何进行分配, 让之前和之后的抖动误差有较好的平衡性和连续性。

如何在不同的建模和渲染场景中生成不同的合适的 Blue noise 是一个重要的话题, 在几乎所有的图像处理场景中 Dithering 都是必要的。

4.3.2 Filtering

Filtering 和图像的特征处理高度相关, 最广泛和最简单的应用是 Blurring 模糊。让高频信息逐渐减少, 剩下的集中在低频信息。基本的 Blurring Filter 使用模糊核将图像的局部进行适当的归一化。

最简单的模糊核是一个 3*3 矩阵, 无限维的卷积核相当于连续空间上的函数卷积

$$h(t) = f * g = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

相当于首先镜像然后向右平移。离散情况中卷积核在二维的像素图像上进行移动, 常用的卷积核 Laplace Kernel 在二维图像上卷积运算的数学表达如下, 其中 h 为卷积核, b 为输出, a 为输入。

$$b[x, y] = \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} a[u, v]h[x - u, y - v]$$

Filtering 方法可以进行图像特征提取, 可以进行竖直方向和水平方向的有限差分, 从而提取不同的信息。边提取算子的数学表达如下:

$$\nabla a = \left(\frac{\partial a}{\partial x}, \frac{\partial a}{\partial y} \right), \quad |\nabla a| = \sqrt{\left(\frac{\partial a}{\partial x} \right)^2 + \left(\frac{\partial a}{\partial y} \right)^2}$$

将以上计算梯度的梯度信息取整数近似可以得到 Sobel edge Filter，是一个离散的非线性卷积核。

$$\frac{\partial}{\partial x} \Rightarrow \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

有限差分可以解决一阶微分，同时可以用 Laplace 算子求二阶微分提取出图像的边缘。离散化的 laplace 算子意思是显然的，就是对比某个像素点和周边的差异。laplace 算子的连续近似如下：

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\begin{aligned} \frac{\partial^2 f}{\partial x^2} &\approx f(x+1, y) - 2f(x, y) + f(x-1, y) \\ \frac{\partial^2 f}{\partial y^2} &\approx f(x, y+1) - 2f(x, y) + f(x, y-1) \end{aligned}$$

对应的离散卷积核为：

$$\mathbf{L} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

4.3.3 Image inpainting

图像补全的中心任务是填补 image 的某些缺失部分，让结果尽可能合理。这里的合理 (Plausible) 有以下含义：

- Spatial locality：和周围和谐
- Occam's Razor：不添加额外的非必要信息
- 梯度尽可能小

这种算法和计算过程常常称为 laplace 图像补全。这里优化函数的数学表示是

$$f = \arg \min_f \iint \|\nabla f\| d\Omega$$

这种技术还被应用于 Image cloning/insertion，图像克隆和拼接，算法可以完成前景和背景的融合。相比于之前的 Laplace 处理，这里要应用 Poisson 处理，目标是将另一张图片注入另一张图片，优化函数令当前的梯度和图片尽可能接近。前面的 Laplace 处理是 $\nabla g = 0$ 的特例 (背景是泛函分析中函数的变分法)

$$Purpose : \min_f \iint_{\Omega} \|\nabla f - \nabla g\|^2 dx dy$$

在计算机科学中本质是离散意义上 Poisson 和 Laplace 方程的求解。将梯度转换为差分，得到以下方程：

$$\frac{f_{i+1,j} + f_{i-1,j} - 2f_{i,j}}{h^2} + \frac{f_{i,j+1} + f_{i,j-1} - 2f_{i,j}}{h^2} = \rho^2$$

证明只需使用二元函数的 Taylor 展开，使用差分的方式进行计算，保留一次项。在实际应用中会注意到边界条件，如果边界条件是已知量（Dirichlet Boundary）则直接带入。离散化的 Laplace 算子形如如下表达式：

$$\nabla^2 f = \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2}$$

对图片中所有的像素点使用 Laplace 算子，得到一个大型的线性方程组。使用 Jacobi 行列式进行迭代求解（Laplace Smoothing 算法）求解矩阵方程 $\mathbf{Ax} = \mathbf{b}$ ，有直接法（LU 分解），Jacobi 迭代法，Gauss-Seidel 迭代法。

Matting Matting 就是一般所指的抠图。抠图使用的方法使用掩码和前景。已知 Alpha 算子对图像进行混合，需要区分前景和背景。混合方程（compositing equation）为：

$$I_i = \alpha_i F_i + (1 - \alpha_i) B_i$$

对 RGB 三通道分别成立：

$$I_i^R = \alpha_i F_i^R + (1 - \alpha_i) B_i^R$$

$$I_i^G = \alpha_i F_i^G + (1 - \alpha_i) B_i^G$$

$$I_i^B = \alpha_i F_i^B + (1 - \alpha_i) B_i^B$$

其中 I_i 为观测像素值， F_i 为前景色， B_i 为背景色， $\alpha_i \in [0, 1]$ 为 Alpha 掩码。每个像素只有 3 个方程却有 F_i, B_i, α_i 共 7 个未知数。所以这个方程有无穷多个解。一种解决方法就是使用绿幕，令背景能够用已知的数值直接带入，减少未知数。这个方法的问题是，绿幕/蓝幕抠图由于漫反射可能会影响到前景。在当代也可以用 AI 方法辅助解决。